

#1. Install libraries

```
!pip install -q datasets transformers accelerate sentencepiece scikit-learn seaborn
```

#2. Import libraries

```
import torch
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from tqdm import tqdm
from datasets import load_dataset
from transformers import pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
```

3. Check GPU

```
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
CUDA available: True
GPU: Tesla T4
```

4. Load Twitter dataset

```
twitter = load_dataset("tweet_eval", "sentiment")

print(twitter)
print("Train labels:", set(twitter["train"]["label"]))
print("Test labels:", set(twitter["test"]["label"]))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
README.md:      23.9k/? [00:00<00:00, 2.57MB/s]

sentiment/train-00000-of-00001.parquet: 100%          3.78M/3.78M [00:02<00:00, 3.99MB/s]

sentiment/test-00000-of-00001.parquet: 100%          901k/901k [00:01<00:00, 1.62MB/s]

sentiment/validation-00000-of-00001.parq(...): 100%    167k/167k [00:00<00:00, 761kB/s]

Generating train split: 100%          45615/45615 [00:00<00:00, 645925.65 examples/s]

Generating test split: 100%          12284/12284 [00:00<00:00, 637950.92 examples/s]

Generating validation split: 100%      2000/2000 [00:00<00:00, 159843.90 examples/s]
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 45615
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 12284
  })
  validation: Dataset({
    features: ['text', 'label'],
    num_rows: 2000
  })
})
Train labels: {0, 1, 2}
Test labels: {0, 1, 2}
```

5. Convert Twitter to binary sentiment

We remove neutral because the task is binary sentiment classification.

```
def convert_twitter_label(example):
    if example["label"] == 0:
        example["label"] = 0 # negative
    elif example["label"] == 2:
        example["label"] = 1 # positive
    else:
        example["label"] = -1 # neutral, remove later
    return example

twitter_binary = twitter.map(convert_twitter_label)
twitter_binary = twitter_binary.filter(lambda x: x["label"] != -1)
```

```
print("Binary train labels:", set(twitter_binary["train"]["label"]))
print("Binary test labels:", set(twitter_binary["test"]["label"]))
print(twitter_binary)
```

Map: 100%45615/45615 [00:01<00:00, 27711.10 examples/s]

Map: 100%12284/12284 [00:00<00:00, 26765.92 examples/s]

Map: 100%2000/2000 [00:00<00:00, 23225.95 examples/s]

Filter: 100%45615/45615 [00:00<00:00, 354785.02 examples/s]

Filter: 100%12284/12284 [00:00<00:00, 271189.92 examples/s]

Filter: 100%2000/2000 [00:00<00:00, 107510.42 examples/s]

Binary train labels: {0, 1}

Binary test labels: {0, 1}

DatasetDict({
 train: Dataset({
 features: ['text', 'label'],
 num_rows: 24942
 })
 test: Dataset({
 features: ['text', 'label'],
 num_rows: 6347
 })
 validation: Dataset({
 features: ['text', 'label'],
 num_rows: 1131
 })
})

6. Select test size

```
TEST_SIZE = 2000

twitter_test = twitter_binary["test"].select(range(TEST_SIZE))

X_test_twitter = twitter_test["text"]
y_test_twitter = twitter_test["label"]

print("Twitter test size:", len(X_test_twitter))
print(twitter_test.column_names)
```

Twitter test size: 2000
['text', 'label']

7. Load recent zero-shot DeBERTa model

```
zero_shot_model = "cross-encoder/nli-deberta-v3-large"

zero_shot_classifier = pipeline(
    "zero-shot-classification",
    model=zero_shot_model,
    device=0 if torch.cuda.is_available() else -1
)

candidate_labels = ["negative", "positive"]
```

config.json:1.10k/? [00:00<00:00, 88.2kB/s]

model.safetensors: 100%1.74G/1.74G [00:08<00:00, 303MB/s]

Loading weights: 100%394/394 [00:00<00:00, 961.08it/s, Materializing param=pooler.dense.weight]

DebertaV2ForSequenceClassification LOAD REPORT from: cross-encoder/nli-deberta-v3-large

Key | Status | |

-----+-----+--

deberta.embeddings.position_ids | UNEXPECTED | |

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

tokenizer_config.json:1.35k/? [00:00<00:00, 102kB/s]

spm.model: 100%2.46M/2.46M [00:00<00:00, 10.7MB/s]

tokenizer.json:8.66M/? [00:00<00:00, 82.1MB/s]

added_tokens.json: 100%26.0/26.0 [00:00<00:00, 2.89kB/s]

special_tokens_map.json: 100%301/301 [00:00<00:00, 37.4kB/s]

8. Prediction function

```
def predict_zero_shot(texts):
    predictions = []

    for text in tqdm(texts):
        result = zero_shot_classifier(
            text,
            candidate_labels=candidate_labels,
```

```

        multi_label=False
    )

    predicted_label = result["labels"][0]

    if predicted_label == "positive":
        predictions.append(1)
    else:
        predictions.append(0)

    return predictions

```

```

# 9. Evaluate zero-shot DeBERTa on overall Twitter

zero_twitter_predictions = predict_zero_shot(X_test_twitter)

zero_twitter_acc = accuracy_score(y_test_twitter, zero_twitter_predictions)
zero_twitter_prec = precision_score(y_test_twitter, zero_twitter_predictions, zero_division=0)
zero_twitter_rec = recall_score(y_test_twitter, zero_twitter_predictions, zero_division=0)
zero_twitter_f1 = f1_score(y_test_twitter, zero_twitter_predictions, zero_division=0)

print("Zero-shot DeBERTa Twitter Accuracy:", zero_twitter_acc)
print("Zero-shot DeBERTa Twitter Precision:", zero_twitter_prec)
print("Zero-shot DeBERTa Twitter Recall:", zero_twitter_rec)
print("Zero-shot DeBERTa Twitter F1:", zero_twitter_f1)

```

```

0%|          | 10/2000 [00:02<04:22, 7.58it/s]You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency p
100%|██████████| 2000/2000 [03:16<00:00, 10.17it/s]
Zero-shot DeBERTa Twitter Accuracy: 0.9005
Zero-shot DeBERTa Twitter Precision: 0.8914835164835165
Zero-shot DeBERTa Twitter Recall: 0.8439531859557867
Zero-shot DeBERTa Twitter F1: 0.8670674682698731

```

```

# 10. Create overall result table

df_overall = pd.DataFrame([
    {
        "Case": "Overall Twitter",
        "Samples": len(X_test_twitter),
        "Accuracy": zero_twitter_acc,
        "Precision": zero_twitter_prec,
        "Recall": zero_twitter_rec,
        "F1-score": zero_twitter_f1
    }
])

df_overall

```

	Case	Samples	Accuracy	Precision	Recall	F1-score
0	Overall Twitter	2000	0.9005	0.891484	0.843953	0.867067

Special Case Testing

```

# 11. Define helper functions for text categories

def is_short(text):
    return len(text.split()) < 8

def has_slang(text):
    slang_words = [
        "meh", "idk", "smh", "bruh", "ugh", "nah", "yikes",
        "lmao", "rofl", "wtf", "omg",
        "lowkey", "highkey", "deadass", "no cap", "cap",
        "sus", "trash", "fire", "mid",
        "this ain't it", "not it", "big yikes",
        "love that for you", "yeah right", "thanks a lot"
    ]
    text = text.lower()
    return any(word in text for word in slang_words)

def has_negation(text):
    text_lower = text.lower()
    negation_terms = [
        "not", "n't", "never", "no", "nothing", "hardly",
        "barely", "without", "neither", "nor"
    ]
    return any(term in text_lower.split() for term in negation_terms) or "n't" in text_lower

def has_mixed_sentiment(text):
    positive_words = [
        "good", "great", "nice", "love", "amazing",
        "best", "happy", "excellent", "perfect"
    ]

```

```

negative_words = [
    "bad", "worst", "hate", "poor", "terrible",
    "awful", "sad", "trash", "disappointing"
]

text = text.lower()
return any(p in text for p in positive_words) and any(n in text for n in negative_words)

def has_uncertainty(text):
    uncertain_words = [
        "maybe", "perhaps", "i guess", "kind of",
        "sort of", "not sure", "it depends", "probably"
    ]

    text = text.lower()
    return any(word in text for word in uncertain_words)

def has_sarcasm_like(text):
    sarcasm_phrases = [
        "yeah right", "thanks a lot", "great...",
        "just perfect", "love that for you",
        "what a joke", "as if", "sure"
    ]

    text = text.lower()
    return any(phrase in text for phrase in sarcasm_phrases)

def has_emoji_or_symbol(text):
    emoji_symbols = [
        "😄", "😂", "😍", "😞", "😏", "❤️", "💀",
        "🔥", "😬", "👍", "💡", "!"
    ]

    return any(symbol in text for symbol in emoji_symbols)

```

#12. Create short, slang, and ambiguous/negation subsets

```

import random

random.seed(42)

short_idx = [
    i for i, t in enumerate(X_test_twitter)
    if is_short(t)
]

slang_idx = [
    i for i, t in enumerate(X_test_twitter)
    if has_slang(t)
]

amb_idx = [
    i for i, t in enumerate(X_test_twitter)
    if (
        has_negation(t)
        or has_mixed_sentiment(t)
        or has_uncertainty(t)
        or has_sarcasm_like(t)
    )
]

BALANCED_SIZE = min(len(short_idx), len(slang_idx), len(amb_idx))

short_idx_balanced = random.sample(short_idx, BALANCED_SIZE)
slang_idx_balanced = random.sample(slang_idx, BALANCED_SIZE)
amb_idx_balanced = random.sample(amb_idx, BALANCED_SIZE)

X_short = [X_test_twitter[i] for i in short_idx_balanced]
y_short = [y_test_twitter[i] for i in short_idx_balanced]

X_slang = [X_test_twitter[i] for i in slang_idx_balanced]
y_slang = [y_test_twitter[i] for i in slang_idx_balanced]

X_amb = [X_test_twitter[i] for i in amb_idx_balanced]
y_amb = [y_test_twitter[i] for i in amb_idx_balanced]

print("Short sample size:", len(X_short))
print("Slang sample size:", len(X_slang))
print("Ambiguous/Negation sample size:", len(X_amb))

```

```

Short sample size: 164
Slang sample size: 164
Ambiguous/Negation sample size: 164

```

13. Evaluate special cases

```
def evaluate_subset(name, texts, labels):
    if len(texts) == 0:
        return {
            "Case": name,
            "Samples": 0,
            "Accuracy": None,
            "Precision": None,
            "Recall": None,
            "F1-score": None
        }

    preds = predict_zero_shot(texts)

    return {
        "Case": name,
        "Samples": len(texts),
        "Accuracy": accuracy_score(labels, preds),
        "Precision": precision_score(labels, preds, zero_division=0),
        "Recall": recall_score(labels, preds, zero_division=0),
        "F1-score": f1_score(labels, preds, zero_division=0)
    }

special_results = []

special_results.append(evaluate_subset("Short", X_short, y_short))
special_results.append(evaluate_subset("Slang", X_slang, y_slang))
special_results.append(evaluate_subset("Ambiguous/Negation", X_amb, y_amb))

df_special = pd.DataFrame(special_results)
df_special
```

100% 164/164 [00:16<00:00, 9.95it/s]
100% 164/164 [00:16<00:00, 10.21it/s]
100% 164/164 [00:15<00:00, 10.49it/s]

	Case	Samples	Accuracy	Precision	Recall	F1-score	
0	Short	164	0.878049	0.939394	0.869159	0.902913	
1	Slang	164	0.914634	0.933333	0.848485	0.888889	
2	Ambiguous/Negation	164	0.914634	0.815789	0.815789	0.815789	

Next steps: [Generate code with df_special](#) [New interactive sheet](#)

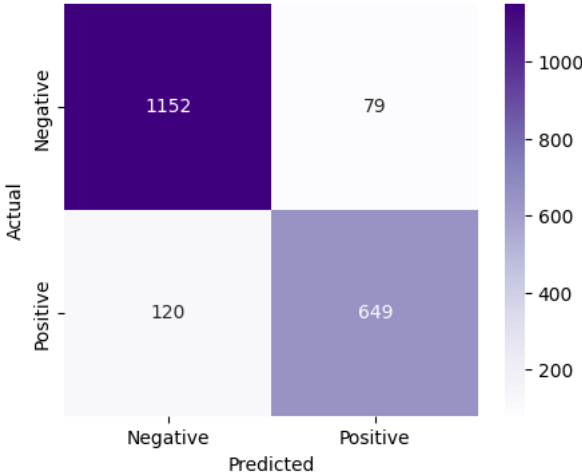
14. Plot confusion matrix

```
cm_zero = confusion_matrix(y_test_twitter, zero_twitter_predictions)

plt.figure(figsize=(5,4))
sns.heatmap(
    cm_zero,
    annot=True,
    fmt="d",
    cmap="Purples",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)

plt.title("Zero-shot DeBERTa-v3-large Confusion Matrix on Twitter")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Zero-shot DeBERTa-v3-large Confusion Matrix on Twitter



16. Create error dataframe

```
error_rows = []

for i, (text, true_label, pred_label) in enumerate(
    zip(X_test_twitter, y_test_twitter, zero_twitter_predictions)
):
    if true_label != pred_label:
        error_rows.append({
            "index": i,
            "text": text,
            "true_label": "positive" if true_label == 1 else "negative",
            "predicted_label": "positive" if pred_label == 1 else "negative"
        })

df_errors = pd.DataFrame(error_rows)

print("Total errors:", len(df_errors))
df_errors.head(30)
```

Total errors: 199

	index	text	true_label	predicted_label	
0	44	@user Wow...this is one deadly statement! Remi...	positive	negative	
1	47	Drop 75% Price: £7.88. Paperback: The Gay & Le...	negative	positive	
2	51	Suarez on twitter: "YNWA #StevenGerrard" [@Lui...	positive	negative	
3	55	@user sadly, Putin has won big time over the p...	positive	negative	
4	60	@user @user oh my godddd!@@@	positive	negative	
5	67	Fascinating interview with former Georgia Pres...	positive	negative	
6	72	Don't forget to buy crap! Also who's got my #H...	negative	positive	
7	73	4. One last time - #ArianaGrandehttps://t.co/0...	positive	negative	
8	83	@user @user @user America was saved. Wake the ...	negative	positive	
9	90	Even after the 4th time watching #FantasticBea...	positive	negative	
10	102	There could be a thousand more Hairspray remak...	positive	negative	
11	115	Westworld and our increasingly wired sense of ...	positive	negative	
12	140	@user @user @user Great, so how far will you c...	negative	positive	
13	144	#FantasticBeasts at @user then to DHS for #Jin...	positive	negative	
14	146	Spirit of my silence i can hear you. But im af...	positive	negative	
15	149	The Democrats are behind the Anti Trump protes...	negative	positive	
16	163	See you kids tomorrow #blackfriday #blackweek ...	positive	negative	
17	177	Watch Tesla Model S P85D's instant speed avoid...	positive	negative	
18	188	VAMOS #LM10 No one can hold you down. If you r...	positive	negative	
19	195	Dear NO voters, are you celebrating your victo...	negative	positive	
20	204	@user @user How do you explain this?	negative	positive	
21	205	saw david beckham on david blaine's tv special...	positive	negative	
22	223	When I'm in public and pull a pair of underwea...	positive	negative	
23	224	My new pet#thewalkingdead	positive	negative	
24	235	Late to this but incredible Trump/Bannon insig...	positive	negative	
25	243	Jill STEIN WHY YOU DO ME A FAVOR GO RECOUNT FI...	negative	positive	
26	250	Im still trying to get over Leonard Cohen dyn...	positive	negative	
27	260	Remarkable. Expert who investigated death of L...	negative	positive	
28	275	Kim Fatty the Third 🤪🤪🤪🤪 China has excelled it...	negative	positive	
29	279	Join #PatriotForceOne to fight progressives co...	positive	negative	

Next steps: [Generate code with df_errors](#) [New interactive sheet](#)

17. Add error category tags

```
df_errors["short"] = df_errors["text"].apply(is_short)
df_errors["slang"] = df_errors["text"].apply(has_slang)
df_errors["negation"] = df_errors["text"].apply(has_negation)
df_errors["mixed_sentiment"] = df_errors["text"].apply(has_mixed_sentiment)
df_errors["sarcasm_like"] = df_errors["text"].apply(has_sarcasm_like)
df_errors["emoji_symbols"] = df_errors["text"].apply(has_emoji_or_symbol)

df_errors.head(30)
```

	index	text	true_label	predicted_label	short	slang	negation	mixed_sentiment	sarcasm_like	emoji_symbols
0	44	@user Wow...this is one deadly statement! Remi...	positive	negative	False	False	False	False	False	True
1	47	Drop 75% Price: £7.88. Paperback: The Gay & Le...	negative	positive	False	False	False	False	False	False
2	51	Suarez on twitter: "YNWA #StevenGerrard" [@Lui...	positive	negative	True	False	False	False	False	False
3	55	@user sadly, Putin has won big time over the p...	positive	negative	False	False	False	False	False	False
4	60	@user @user oh my godddd!@@@	positive	negative	True	False	False	False	False	True
5	67	Fascinating interview with former Georgia Pres...	positive	negative	False	False	False	False	False	False
6	72	Don't forget to buy crap! Also who's got my #H...	negative	positive	False	False	True	False	False	True
7	73	4. One last time - #ArianaGrandehttps://t.co/0...	positive	negative	True	False	False	False	False	False
8	83	@user @user @user America was saved. Wake the ...	negative	positive	False	False	False	False	False	False
9	90	Even after the 4th time watching #FantasticBea...	positive	negative	False	False	False	False	False	True
10	102	There could be a thousand more Hairspray remak...	positive	negative	False	False	False	False	False	False
11	115	Westworld and our increasingly wired sense of ...	positive	negative	False	False	False	False	False	False
12	140	@user @user @user Great, so how far will you c...	negative	positive	False	False	False	False	False	False
13	144	#FantasticBeasts at @user then to DHS for #Jin...	positive	negative	False	False	False	False	False	False
14	146	Spirit of my silence i can hear you. But im af...	positive	negative	False	True	False	False	False	False
15	149	The Democrats are behind the Anti Trump protes...	negative	positive	False	False	False	False	False	False
16	163	See you kids tomorrow #blackfriday #blackweek ...	positive	negative	False	False	False	False	False	False
17	177	Watch Tesla Model S P85D's instant speed avoid...	positive	negative	False	False	False	False	False	False
18	188	VAMOS #LM10 No one can	positive	negative	False	False	True	False	False	False

Next steps:

[Generate code with df_errors](#)

[New interactive sheet](#)

```
# 18. Count error categories
error_category_summary = pd.DataFrame({
    "Category": [
        "Short",
        "Slang",
        "Negation",
        "Mixed sentiment",
        "Sarcasm-like",
        "Emoji/symbols"
    ],
    "Error Count": [
        df_errors["short"].sum(),
        df_errors["slang"].sum(),
        df_errors["negation"].sum(),
        df_errors["mixed_sentiment"].sum(),
        df_errors["sarcasm_like"].sum(),
        df_errors["emoji_symbols"].sum()
    ]
})

error_category_summary
```

	Category	Error	Count	
0	Short		20	
1	Slang		14	
2	Negation		34	
3	Mixed sentiment		5	
4	Sarcasm-like		2	
5	Emoji/symbols		35	

Next steps:

[Generate code with error_category_summary](#)

[New interactive sheet](#)

```
# 19. Show error examples

pd.set_option("display.max_colwidth", 200)

df_errors[
    ["text", "true_label", "predicted_label", "short", "slang", "negation", "mixed_sentiment", "sarcasm_like", "emoji_symbols"]
].head(20)
```

	text	true_label	predicted_label	short	slang	negation	mixed_sentiment	sarcasm_like	emoji_symbols
0	@user Wow...this is one deadly statement! Reminding me of the Greta Muhammad Ali.	positive	negative	False	False	False	False	False	True
1	Drop 75% Price: £7.88. Paperback: The Gay & Lesbian Marriage & Family Reader: Analyses of Problems and Prospe...	negative	positive	False	False	False	False	False	False
2	Suarez on twitter: "YNWA #StevenGerrard" [@LuisSuarez9] #FCBlive	positive	negative	True	False	False	False	False	False
3	@user sadly, Putin has won big time over the past 10-15 years. His biggest triumph being the election of his proxy @user	positive	negative	False	False	False	False	False	False
4	@user @user oh my godddd!@@@	positive	negative	True	False	False	False	False	True
5	Fascinating interview with former Georgia President Mikheil Saakashvili - very candid about Putin, Trump & NATO	positive	negative	False	False	False	False	False	False
6	Don't forget to buy crap! Also who's got my #Hatchimals hookup	negative	positive	False	False	True	False	False	True
7	4. One last time - #ArianaGrandehttps://t.co/0o4T0q3GsY	positive	negative	True	False	False	False	False	False
8	@user @user @user America was saved. Wake the hell up. You want 2M refugees? Global gov?TPP?Learn the facts goto college	negative	positive	False	False	False	False	False	False
	Even after the 4th time watching #FantasticBeasts I still turned up at the								